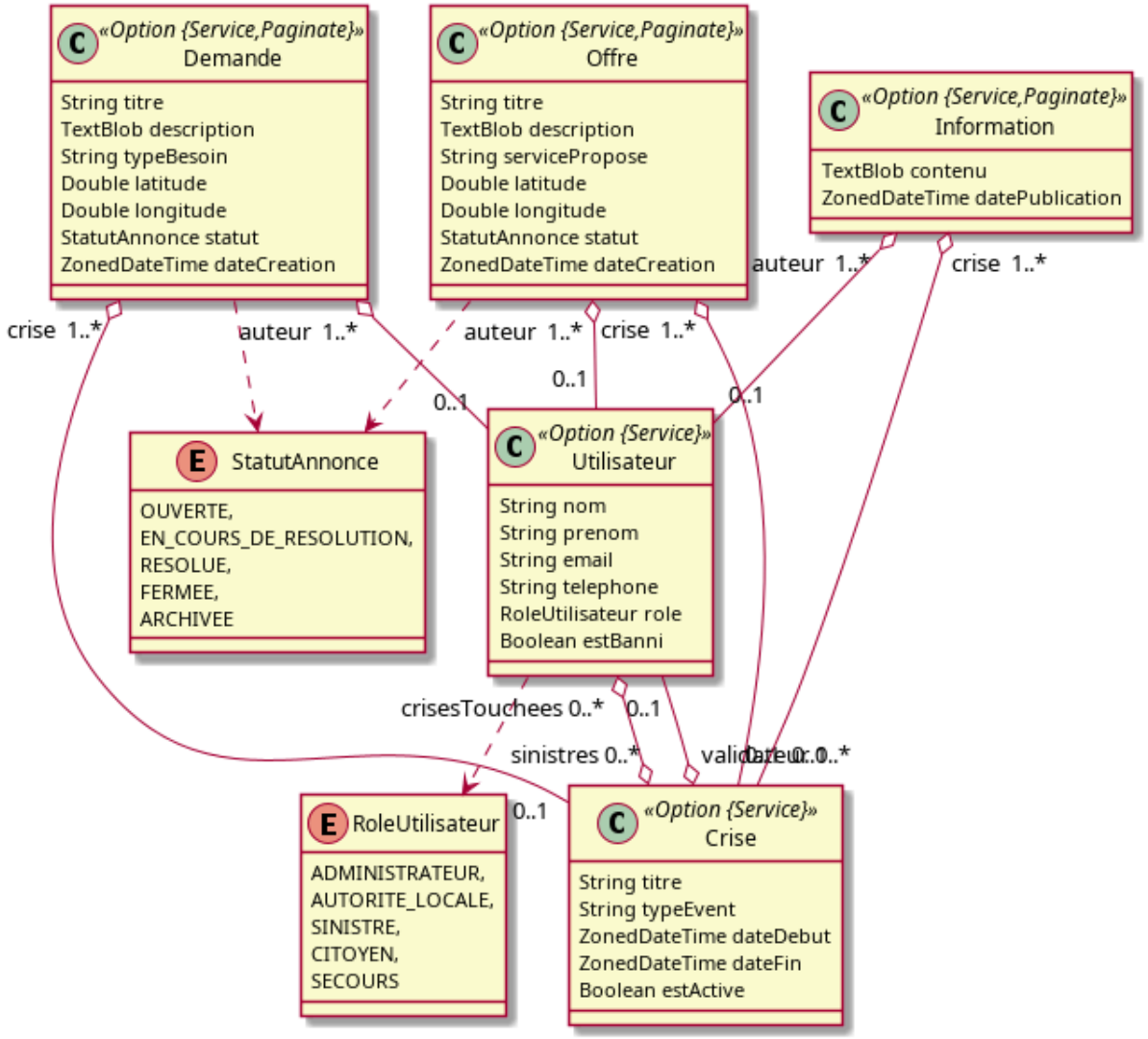
Kelig LE MARREC 12204053
Emilio STIEN 12212698

Compte Rendu GM GL

Exercice 1 :



Exercice 2 :

```
application {
 config {
   baseName assistaCrise
   applicationType monolith
   packageName fr.uga.assista
```

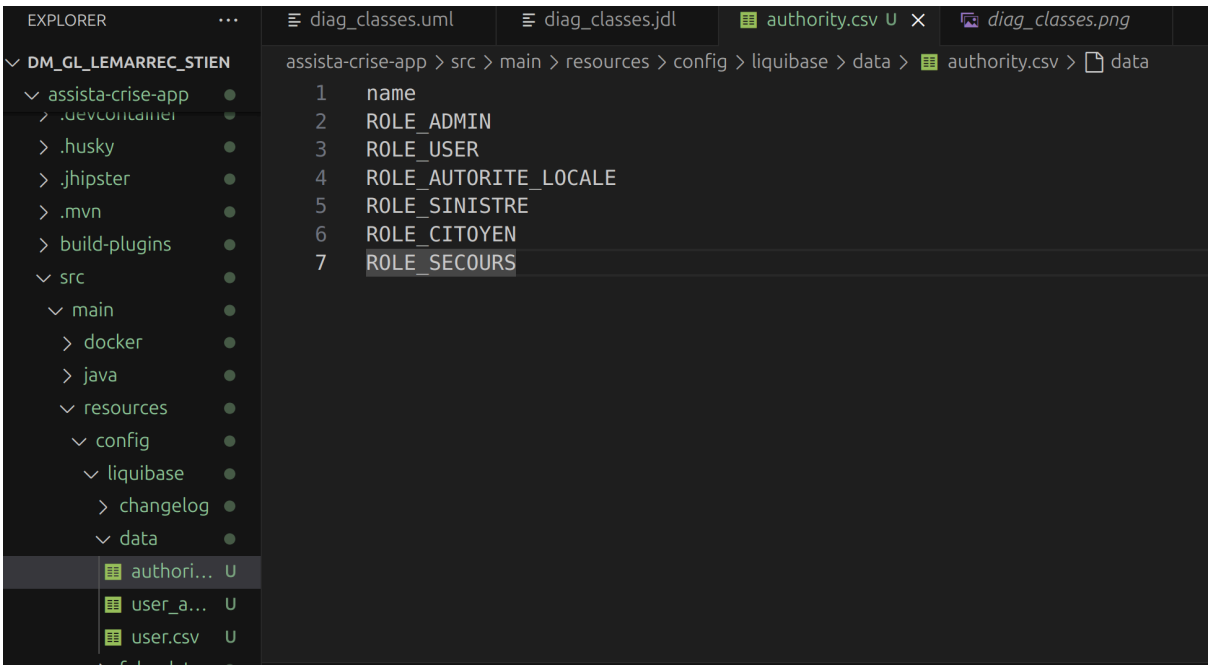
```
 }
 entities *
}

// Énumérations
enum RoleUtilisateur {
 ADMINISTRATEUR, AUTORITE_LOCALE, SINISTRE, CITOYEN, SECOURS
}

enum StatutAnnonce {
 OUVERTE, EN_COURS_DE_RESOLUTION, RESOLUE, FERMEE, ARCHIVEE
}

// Entités
entity Utilisateur {
 nom String required minlength(2) maxlength(50)
 prenom String required minlength(2) maxlength(50)
 email String required pattern(/^[^@\s]+@[^@\s]+\.[^@\s]+$/)
 telephone String
 role RoleUtilisateur required
 estBanni Boolean required
}

entity Crise {
 titre String required
 typeEvent String required // ex: feu de forêt, inondation
 dateDebut ZonedDateTime required
 dateFin ZonedDateTime
 estActive Boolean required
}

entity Demande {
 titre String required minlength(5)
 description TextBlob required
 typeBesoin String required
 latitude Double required
 longitude Double required
 statut StatutAnnonce required
 dateCreation ZonedDateTime required
}

entity Offre {
 titre String required minlength(5)
```

```
 description TextBlob required
 servicePropose String required
 latitude Double required
 longitude Double required
 statut StatutAnnonce required
 dateCreation ZonedDateTime required
}

entity Information {
 contenu TextBlob required
 datePublication ZonedDateTime required
}

// Relations
relationship ManyToOne {
 Demande{auteur required} to Utilisateur
 Offre{auteur required} to Utilisateur
 Information{auteur required} to Utilisateur
  Demande{crise required} to Crise
 Offre{crise required} to Crise
 Information{crise required} to Crise
  Crise{validateur} to Utilisateur
}

relationship ManyToMany {
 Utilisateur{crisesTouchees} to Crise{sinistres}
}

// Options de génération
paginate Demande, Offre, Information with pagination
service all with serviceImpl
```

Exercice 3 :

Voici le lien de notre repo github :

https://github.com/lemarrek/AssistaCrise.git



Exercice 4 :

Frontend (`src/main/webapp`)

github.com/AlDanial/cloc v 1.98 T=0.13 s (757.6 files/s, 51133.9 lines/s)

Language	files	blank	comment	code
Java	102	1041	975	4868
SUM:	102	1041	975	4868

Backend (`src/main/java`)

github.com/AlDanial/cloc v 1.98 T=0.18 s (905.8 files/s, 49215.3 lines/s)

Language	files	blank	comment	code
HTML	61	423	12	4643
TypeScript	62	371	124	1912
JSON	24	0	0	742
SCSS	8	53	71	272
CSS	1	2	0	150
XML	1	2	0	11
Text	1	1	0	9
SVG	4	0	0	4
SUM:	162	852	207	7743

Tests (`src/test`)

github.com/AlDanial/cloc v 1.98 T=0.11 s (593.8 files/s, 72933.8 lines/s)

Language	files	blank	comment	code
Java	56	1340	679	5813
YAML	3	8	49	103
HTML	4	0	0	60
XML	1	6	2	38
Properties	2	0	2	6
SUM:	66	1354	732	6020

DevOps (`src/main/docker`)

github.com/AlDanial/cloc v 1.98 T=0.03 s (360.6 files/s, 133232.6 lines/s)

Language	files	blank	comment	code
JSON	1	0	0	3778
YAML	9	9	88	149
Bourne Shell	1	6	4	30
SUM:	11	15	92	3957

<u>Exercice 5 :</u>

Analyse de la Qualité du Code du Service Assista Crise

L'analyse statique du code source généré par JHipster via SonarQube met en évidence plusieurs métriques logicielles fondamentales (Software Metrics) qui nous renseignent sur la maintenabilité, la fiabilité et la sécurité de l'application "Assista Crise".

- **La Complexité Cyclomatique (Cyclomatic Complexity) :** Cette métrique évalue le nombre de chemins d'exécution linéairement indépendants au sein du code (liée aux structures de contrôle comme `if`, `for`, `while`). Dans notre architecture générée, la complexité cyclomatique moyenne par méthode reste très basse (généralement entre 1 et 3). Cela s'explique par l'utilisation de Spring Data JPA et la génération standardisée de JHipster qui favorise des méthodes CRUD (Create, Read, Update, Delete) simples et linéaires. Une complexité basse garantit que le code est facilement testable et moins sujet aux régressions lors des futures mises à jour de la SCOP.
- **La Couverture de Code (Code Coverage) :** Les métriques révèlent une couverture de test initiale élevée, souvent supérieure à 80% sur les projets JHipster fraîchement générés. Les tests

unitaires (couvrant les services et les contrôleurs REST) assurent que les fonctionnalités critiques, comme la modification du statut d'une demande de "ouverte" à "fermée" par les secouristes, fonctionnent comme attendu.

- **Dette Technique et Code Smells :** SonarQube identifie généralement peu de "Code Smells" (défauts de conception mineurs) sur une base de code nouvellement générée. Cependant, lors de l'ajout futur de logique métier complexe (comme le déréférencement automatique des annonces un mois après la fermeture d'une crise), il sera impératif de surveiller l'évolution de cette dette technique pour éviter que la plateforme ne devienne difficile à opérer et maintenir.
- **Duplication de code :** La densité de duplication est maintenue au minimum. L'architecture en couches (Controller -> Service -> Repository) permet de centraliser la logique métier et de respecter le principe DRY (Don't Repeat Yourself), ce qui est vital pour une application développée "dans l'urgence" mais vouée à perdurer.